Name	Default	Description
`RequireDigit`	`true`	Specifies whether the password must include a digit.
`RequiredLength`	`6`	Specifies the minimum number of characters in a password.
`RequireLowercase`	`true`	Determines whether passwords must contain a lowercase ASCII character.
`RequireNonAlphanumeric`	`true`	Indicates whether passwords must contain a non-alphanumeric character.
`RequiredUniqueChars`	`1`	Specifies the number of unique characters required in the password.
`RequireUppercase`	`true`	If true, the password must contain an uppercase ASCII character.

Within Identity, passwords are hashed before they are stored. The hashing process also includes a *salt*—a randomly generated value that is added to the password to ensure the resulting hash for two identical passwords is unique. Then the salt and the hashed password are stored together in the `PasswordHash` column of the `AspNetUsers` table.

When a user submits a password as part of the login process, the `PasswordHash` value is retrieved for the username, and the salt is extracted and used to hash the submitted password. Hashing is deterministic in that, given the same input, hashing will always result in the same output. So the salted and hashed version of the submitted password should match what is stored in the database, assuming the correct password was submitted. Hashing is also one-way, unlike encryption. You cannot reverse a hashed value to retrieve the original value.

This might sound complicated, and the algorithms that take care of processing the password for storage certainly are, which emphasizes the point that you should not attempt to replace the existing implementation, unless you know what you are doing.